

Schemi XML

Prof. **Marco Camurri**

Cos'è un *schema* in XML?

Uno **schema** è un file in cui si descrive formalmente la *struttura* che deve essere rispettata da tutti i documenti appartenenti allo stesso linguaggio XML.

- Lo schema contiene le regole necessarie per determinare se un particolare documento è valido in un dato linguaggio XML.
- Lo schema stabilisce:
 - 1) qual è il nome dell'elemento radice
 - 2) quali nomi di elemento sono ammessi
 - 3) quali elementi figli sono ammessi per un dato elemento padre
 - 4) quali attributi sono ammessi per un dato elemento, e quali di questi sono obbligatori
 - 5) quali valori sono ammessi per un dato attributo

DTD e XML-Schema

Attualmente esistono due linguaggi alternativi per la definizione di schemi:

■ Linguaggio DTD

- acronimo di ***Document Type Definition***
- meno recente
- non basato su XML

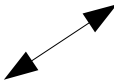
■ Linguaggio XML-Schema (XSD)

- Detto anche **linguaggio XSD** (***XML-Schema Definition***)
- i file scritti in questo linguaggio hanno estensione *.xsd*
- più recente e più espressivo rispetto a DTD
- basato su XML

Esempio di schema in linguaggio DTD

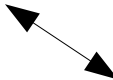
```
<!ELEMENT messaggio (da,a,testo)>  
<!ELEMENT da (#PCDATA)>  
<!ELEMENT a (#PCDATA)>  
<!ELEMENT testo (#PCDATA)>
```

contenuto del
file
schema.dtd



```
<?xml version="1.0"?>  
<messaggio>  
  <da>Mario</da>  
  <a>Anna</a>  
  <testo>ciao!</testo>  
</messaggio>
```

**esempio di documento
VALIDO rispetto allo
schema**



```
<?xml version="1.0"?>  
<messaggio>  
  <da>Mario</da>  
  <corpo>ciao!</corpo>  
</messaggio>
```

**esempio di documento
NON VALIDO rispetto allo
schema**

Esempio di schema in linguaggio XSD

- L'esempio precedente, riscritto in linguaggio XML-Schema, diventa:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="messaggio">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="da" type="xs:string"/>
        <xs:element name="a" type="xs:string"/>
        <xs:element name="testo" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Correttezza di un documento XML

- Esistono due livelli di "correttezza" per un documento XML. Infatti un documento XML può essere semplicemente *ben-formato*, oppure *valido*.
- Un documento XML si dice **ben-formato** (*well-formed*) se rispetta la sintassi generale di XML:
 - unico elemento radice
 - chiusura di tutti gli elementi
 - valore tra virgolette per tutti gli attributi
 - ecc.
- Un documento XML si dice **valido rispetto ad uno schema** se è ben-formato e rispetta tutte le regole contenute nello schema.
- La parola *valido* quindi ha senso solo in presenza di uno schema.

Perchè scrivere uno schema?

- La struttura di uno documento XML potrebbe essere descritta anche in linguaggio naturale:

tutti i file XML validi del mio linguaggio devono avere un tag radice di nome "messaggio", contenente quattro elementi figli di nome "da" , "a" , "testo",

- Produrre una descrizione formale della struttura, in un linguaggio come DTD o XSD, ci consente di automatizzare il processo di validazione, poichè ci consente di utilizzare un validatore ...

Validatori

Un **validatore XML** è un programma che riceve in input:

- un file XML
- uno schema (DTD o XSD)

e determina se il file XML è valido rispetto allo schema.

- sono disponibili numerosi validatori online (vedi slide seguente)
- in molti linguaggi di programmazione sono disponibili librerie che permettono, con poche righe di codice, di richiamare un validatore

Validatori online

- <https://www.freeformatter.com/xml-validator-xsd.html>
- <https://www.liquid-technologies.com/online-xsd-validator>
- <http://www.utilities-online.info/xsdvalidation/>

XML-Schema

- Non affrontiamo lo studio del linguaggio DTD, e iniziamo invece lo studio della sua più moderna alternativa, XML-Schema.
- Riassunto di quando visto finora:
 - XML-Schema è un linguaggio per definire la struttura di un documento XML
 - Un file in linguaggio XML-Schema è un file .xml, a cui per convenzione viene assegnata l'estensione .xsd
 - Un file .xsd consente di definire nuovi linguaggi basati su XML
 - Un file .xsd contiene le definizioni degli elementi che possono comparire nel nostro nuovo linguaggio

Elementi semplici e complessi

XML-Schema prevede due tipi fondamentali di elementi:

Un elemento è **semplice** se non contiene nè elementi figli nè attributi. Gli elementi non semplici si dicono **complessi**.

Esempi:

```
<cognome>Rossi</cognome>
```

cognome è un elemento semplice

```
<auto targa="FG567HF" />
```

auto è un elemento complesso
perchè ha un attributo

```
<persona>  
  <nome>mario</nome>  
</cognome>
```

persona è un elemento complesso
perchè ha un figlio

Elementi semplici

Scriviamo il nostro primo file xsd, in cui definiamo un unico elemento semplice (l'elemento *cognome*)

```
<?xml version="1.0"?>  
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="cognome" type="xs:string" />  
</xs:schema>
```

Elementi semplici

Un file XML rispetta lo schema precedente se e solo se rispetta queste 3 regole:

- 1) contiene un unico elemento, delimitato dai tag **<cognome>** e **</cognome>**
- 2) il contenuto dell'elemento è una stringa semplice (cioè non contiene altri tag annidati)
- 3) non sono presenti attributi

```
<?xml version="1.0"?>  
<cognome>rossi</cognome>
```

Tipi di dato semplici predefiniti

- Nell'esempio precedente abbiamo dichiarato che l'elemento cognome è di tipo "xs:string"
- XML-Schema prevede i seguenti tipi di dato semplici predefiniti (*built-in data types*) :

Tipo di dato	Esempio
xs:string	<code><regista>Steven Spielberg</regista></code>
xs:decimal	<code><prezzo>12.50</prezzo></code>
xs:integer	<code><codice_prodotto>1637</codice_prodotto></code>
xs:date	<code><scadenza>2019-11-26</scadenza></code>
xs:time	<code><ora_partenza>23:30:00</ora_partenza></code>

Tipi di dato semplici predefiniti

Il contenuto di un elemento con tipo "xs:date" deve essere una data valida, nel formato anno-mese-giorno. Esempio:

Definizione dell'elemento:

```
<xs:element name="scadenza" type="xs:date" />
```

Esempio di elemento valido:

```
<scadenza>2019-11-28</scadenza>
```

Esempi di elementi non validi:

```
<scadenza>ciao</scadenza>
```

il contenuto non è una data

```
<scadenza>2019-02-29</scadenza>
```

data non valida

```
<scadenza>28-11-2019</scadenza>
```

formato non valido

Restrizioni

- E' possibile definire nuovi tipi di dato semplice aggiungendo dei vincoli, o *restrizioni*, ai tipi semplici predefiniti
- Sono disponibili diverse tipi di restrizioni:
 - **restrizioni sull'intervallo**: permettono di indicare un valore minimo e/o massimo. Esempio: `<voto>9</voto>`
 - **restrizioni sulla lunghezza**: permettono di indicare la lunghezza minima e/o massima di una stringa. Esempio: `<password>abc</password>`
 - **restrizioni di tipo *pattern***: permettono di indicare uno "schema". Esempio: la targa di un veicolo deve essere una stringa composta da 2 lettere, 3 numeri, e 2 lettere `<targa>AB123HF</targa>`
 - **restrizioni di tipo *enumeration***: permettono di fornire un elenco di valori consentiti. Esempio: `<colore>rosso</colore>`

Restrizioni

- Per definire una restrizione, occorre annidare i tag **simpleType** e **restriction** all'interno dell'elemento da restringere, e scegliere un tipo semplice predefinito da usare come base

```
<xs:element name=".....">
  <xs:simpleType>
    <xs:restriction base=".....">
      .... qui specificare il tipo di restrizione ...
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Esempio: restrizione sul range

- Definiamo l'elemento **voto** in modo che possa contenere solo numeri interi compresi tra 2 e 10

```
<xs:element name="voto">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="2"/>
      <xs:maxInclusive value="10"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

- Esempio di elemento valido rispetto allo schema:

```
<voto>6</voto>
```

Esempio: restrizione sulla lunghezza

- Definiamo l'elemento **password** in modo che possa contenere solo stringhe di lunghezza compresa tra 8 e 20 caratteri

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minLength value="8"/>
      <xs:maxLength value="20"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

- Esempio di elemento valido rispetto allo schema:

```
<password>abc123!!!</password>
```

Esempio: restrizione di tipo *pattern*

- Definiamo l'elemento **targa** in modo che possa contenere solo stringhe nel formato "2 lettere + 3 cifre + 2 lettere"

```
<xs:element name="targa">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:pattern value="[A-Z][A-Z][0-9][0-9][0-9][A-Z][A-Z]"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

- Esempio di elemento valido rispetto allo schema:

```
<targa>AB123DE</password>
```

Espressioni regolari (regex)

- In informatica, una espressione regolare (*regular expression* o *regex*) è un'espressione tramite cui si definisce un insieme di stringhe.
- Ad esempio, all'espressione regolare "[A-Z][A-Z]" corrisponde l'insieme di tutte le possibili stringhe composte da due lettere maiuscole.

Insieme di stringhe valide = { "AA", "AB", "AC", "ZZ" }

- Si dice anche che le stringhe valide producono un ***match*** (corrispondenza) con l'espressione regolare.

Sintassi delle espressioni regolari

■ [] Singolo carattere compreso nel range specificato

Esempi: [A-Z] , [yn] , [A-Za-z0-9]

■ | Or

Esempio: [a-f]|[A-F]|[0-9]

■ * Zero o più occorrenze

Esempio: [abc]*

■ + Una o più occorrenze

■ ? Zero o una occorrenza

■ () Blocco

■ . Un carattere qualsiasi

■ \d Una cifra qualsiasi

Esempio: restrizione di tipo *enumeration*

- Definiamo l'elemento **targa** in modo che possa contenere solo stringhe nel formato "2 lettere + 3 cifre + 2 lettere"

```
<xs:element name="colore">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:enumeration value="rosso"/>  
      <xs:enumeration value="verde"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

- Esempio di elemento valido rispetto allo schema:

```
<colore>rosso</colore>
```

Elementi complessi

- Per definire un elemento che possa contenere elementi figli si usa il seguente schema generale:

```
<xs:element name=".....">  
  <xs:complexType>  
    .....  
  </xs:complexType>  
</xs:element>
```

- All'interno dell'elemento `complexType` occorre poi aprire uno di questi tag: `<xs:sequence>`, `<xs:all>` oppure `<xs:choice>`
- *sequence*, *all* e *choice* sono detti **indicatori**
- Iniziamo con un esempio che mostra il significato dell'indicatore *sequence*

L'indicatore *sequence*

- Definiamo l'elemento auto, che deve contenere esattamente tre elementi figli chiamati modello, porte e targa (in questo ordine)

```
<xs:element name="auto">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="modello" type="xs:string" />
      <xs:element name="porte" type="xs:integer" />
      <xs:element name="targa" type="xs:string" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

L'indicatore *sequence*

Esempi:

```
<auto>  
  <modello>Punto</modello>  
  <porte>5</porte>  
  <targa>AB123DE</targa>  
</auto>
```

VALIDO

```
<auto>  
  <targa>AB123DE</targa>  
</auto>
```

NON VALIDO perchè
mancano elementi

```
<auto>  
  <targa>AB123DE</targa>  
  <modello>Punto</modello>  
  <porte>12345</porte>  
  <marca>FIAT</marca>  
</auto>
```

NON VALIDO perchè gli
elementi non sono
nell'ordine giusto e c'è un
elemento non previsto
(marca)

Numero di occorrenze

- Quando si utilizza l'indicatore *sequence*, è possibile usare gli attributi **minOccurs** e **maxOccurs** nel tag **element** per specificare il numero minimo e massimo di occorrenze dell'elemento nella sequenza.
- Esempio: elemento persona con email facoltativa (minOccurs=0) e massimo 2 email (maxOccurs=2)

```
<xs:element name="persona">  
  <xs:complexType>  
    <xs:sequence>  
      <xs:element name="nome" />  
      <xs:element name="email" minOccurs="0" maxOccurs="2" />  
    </xs:sequence>  
  </xs:complexType>  
</xs:element>
```

Numero di occorrenze

Esempi:

```
<persona>  
  <nome> Mario </nome>  
</person>
```

VALIDO

```
<persona>  
  <nome> Mario </nome>  
  <email> aaaaaaaa </email>  
  <email> bbbbbbbb </email>  
</person>
```

VALIDO

```
<persona>  
  <nome> Mario </nome>  
  <email> aaaaaaaa </email>  
  <email> bbbbbbbb </email>  
  <email> cccccccc </email>  
</person>
```

NON VALIDO
(troppe email)

L'indicatore *all*

- L'indicatore *all* consente di definire elementi complessi in cui gli elementi figli possono comparire in qualunque ordine

- Esempio:

```
<xs:element name="computer">
  <xs:complexType>
    <xs:all>
      <xs:element name="cpu" />
      <xs:element name="ram" />
      <xs:element name="hd" minOccurs="0" />
    </xs:all>
  </xs:complexType>
</xs:element>
```

L'indicatore *choice*

- L'indicatore *choice* consente di definire un elemento complesso in cui può comparire uno solo fra i figli specificati all'interno della scelta
- Esempio:

```
<xs:element name="mezzo">  
  <xs:complexType>  
    <xs:choice>  
      <xs:element name="auto" />  
      <xs:element name="moto" />  
      <xs:element name="bici" />  
    </xs:choice>  
  </xs:complexType>  
</xs:element>
```

Elementi con attributi

- Un elemento con attributi è considerato di tipo complesso
- Esempio: definiamo l'elemento punto con gli attributi x e y obbligatori, e l'attributo z opzionale

```
<xs:element name="punto">  
  <xs:complexType>  
    <xs:attribute name="x" type="xs:decimal" use="required" />  
    <xs:attribute name="y" type="xs:decimal" use="required" />  
    <xs:attribute name="z" type="xs:decimal" />  
  </xs:complexType>  
</xs:element>
```

- Esempio di elemento valido:

```
<punto x="10.5" y="0.9999" />
```

Elementi con attributi e figli

- Per definire un elemento contenente sia attributi, sia figli, occorre definire un elemento complesso che contenga prima l'indicatore (sequence, all o choice) e poi uno o più elementi attribute.

Elementi con attributi e figli

Definizione dell'elemento

```
<xs:element name="raccolta">  
  <xs:complexType>  
    <xs:sequence>  
      <xs:element name="brano" type="xs:string" maxOccurs="5"/>  
    </xs:sequence>  
    <xs:attribute name="autore" type="xs:string" />  
  </xs:complexType>  
</xs:element>
```

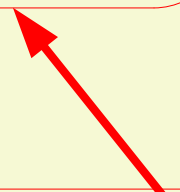
Esempio di elemento valido:

```
<raccolta autore="..." >  
  <brano>...</brano>  
  <brano>...</brano>  
  <brano>...</brano>  
</raccolta>
```

Definizione di tipi personalizzati

E' possibile definire nuovi tipi in modo da semplificare lo schema:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:complexType name="tipo_persona" >
    <xs:sequence>
      <xs:element name="nome" />
      <xs:element name="cognome"/>
    </xs:sequence>
  </xs:complexType>
  <xs:element name="famiglia">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="padre" type="tipo_persona" />
        <xs:element name="madre" type="tipo_persona" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



Definizione di tipi personalizzati

- Esempio di documento valido rispetto allo schema precedente:

```
<?xml version="1.0" encoding="UTF-8"?>
<famiglia>

  <padre>
    <nome></nome>
    <cognome></cognome>
  </padre>

  <madre>
    <nome></nome>
    <cognome></cognome>
  </madre>

</famiglia>
```